

Phase 1: Problem Definition & EDA

- **Business Problem:** Customer churn is a critical issue for banks and subscription-based businesses. Identifying customers who are likely to leave (churn) allows the business to proactively offer incentives to retain them, thereby saving customer acquisition costs and stabilizing revenue.
- **Feature Identification:** * **Numerical Features:** SeniorCitizen, Tenure, MonthlyCharges, TotalCharges
 - **Categorical Features:** Gender, Contract, PaymentMethod
 - *(Note: Irrelevant identifiers like CustomerID and Unnamed: 0 were dropped).*
- **Target Variable Distribution:** The target variable Churn is highly imbalanced. Based on the data, ~73.3% of customers did not churn (Class 0) while only ~26.7% churned (Class 1).
- **Visualizations:** Histograms, boxplots, a correlation heatmap, and a target distribution plot were successfully generated to analyze data spread and multicollinearity.

Phase 2: Data Preprocessing

- **Missing Values:** We verified the data types (ensuring TotalCharges is correctly formatted as numerical) and confirmed no missing values hindered the dataset. We set up an imputation strategy (median replacement) as a safeguard for production environments.

Phase 3: Model Implementation

- **Encoding & Scaling:** Categorical features were transformed using One-Hot Encoding (drop='first' to avoid multicollinearity). Numerical features were standardized using StandardScaler so that features with larger scales (like TotalCharges) do not dominate the model.
- **Data Split:** The data was split into 80% Training and 20% Testing sets, utilizing stratify=y to preserve the original class distribution in both splits.
- **Algorithm & Hyperparameters:** A standard **Logistic Regression** model was trained.
 - penalty='l2': Ridge regularization was chosen to prevent overfitting by penalizing large coefficients.

- $C=1.0$: Standard regularization strength.
- `solver='liblinear'`: Chosen because it is highly efficient and stable for relatively small and standard tabular datasets.

Phase 4: Model Evaluation

The evaluation metrics on the test set reveal the impact of class imbalance:

- **Standard Accuracy:** 73.3%
- **Precision, Recall, and F1 Score:** Precision: 0.00 | Recall: 0.00 | F1 Score: 0.00
- **ROC-AUC Score:** 0.458
- **Insight:** The model predicted "0" (No Churn) for every single instance. While this yields a seemingly decent accuracy of 73% (which is just the baseline majority class ratio), it completely fails to detect the churners.

Phase 5: Interpretation & Insights

- **Analyzing Coefficients:** The logistic regression weights indicate how each feature influences the probability of churn.
 - **Positive Impact (Increases Churn Risk):** `Contract_One year` (+0.079), `Tenure` (+0.050), and `PaymentMethod_Credit card` (+0.037).
 - **Negative Impact (Decreases Churn Risk):** `PaymentMethod_Mailed check` (-0.083), `TotalCharges` (-0.035).

Phase 6: Conclusion & Future Work

- **Key Findings:** While the preprocessing pipeline is robust, the base Logistic Regression model is heavily biased toward the majority class, making it ineffective for practical churn detection without further intervention.
- **Limitations:** The severe class imbalance crippled the model's ability to learn the minority class. Additionally, Logistic Regression assumes a linear relationship between features and the log-odds of the target, which may oversimplify complex customer behaviors.
- **Future Improvements:**
 - **1. Address Class Imbalance:** Implement **SMOTE** (Synthetic Minority Over-sampling Technique) or class weighting (`class_weight='balanced'`) to help the model learn the churn minority class.

- **2. Advanced Algorithms:** Experiment with tree-based ensemble methods like **Random Forest** or **XGBoost**, which naturally handle non-linear relationships and often perform better on imbalanced data.
- **3. Feature Engineering:** Create interaction terms (e.g., MonthlyCharges / Tenure) to capture deeper billing behaviors.